
HIDING GLOBAL SYNCHRONIZATION LATENCY IN 3D STENCIL SOLVERS VIA PIPELINED KRYLOV METHODS

Anteneh Getachew Yitayal
(256983)
Data Science
University of Trento
Trento, Italy

Rares Andrei
(265214)
Cyber Security
University of Trento
Trento, Italy

ABSTRACT

Global synchronization is a major bottleneck for scaling standard Conjugate Gradient (CG) methods on modern HPC architectures. This work evaluates two Jacobi-preconditioned CG (JCG) solvers: a baseline utilizing global vector replication via `MPI_Allgatherv` and a pipelined JCG designed for communication–computation overlap. Using a 27-point stencil and Z-slab decomposition, we benchmark grids up to 1100^3 (1.33 billion unknowns) on 384 MPI ranks. Results show the baseline is dominated by communication overhead (>90% of iteration time) and hits a "memory wall" at large scales. Conversely, the pipelined JCG successfully masks global reduction and halo exchange latencies within SpMV memory stalls, justified by a low operational intensity of 0.22 FLOPs/Byte. Under strong scaling, the pipelined variant maintains 87.8% efficiency on four nodes relative to its single-node performance and delivers an order-of-magnitude speedup over the baseline. We conclude that for billion-unknown problems, pipelining is a structural requirement that transforms latency-bound solvers into bandwidth-efficient implementations.

Keywords Parallel Computing, Conjugate Gradient, MPI, Pipelined Krylov Methods, Communication-Hiding, Scalability, 3D Stencils.

1 Introduction

The solution of large sparse symmetric positive definite (SPD) linear systems

$$\mathbf{Ax} = \mathbf{b}$$

is a fundamental challenge in scientific computing, arising in fields such as computational fluid dynamics and structural analysis. While the Conjugate Gradient (CG) method is the standard iterative solver for such systems, its performance on modern HPC architectures is often hindered by global synchronization bottlenecks and numerical stagnation in large-scale problems [11].

To address these challenges, this project evaluates two variants of the Conjugate Gradient method using a 3D z-slab domain decomposition. Both variants utilize Jacobi preconditioning, which is chosen for its minimal computational overhead, lack of memory fill-in, and trivial parallelization in distributed-memory environments. The primary objective is to analyze the trade-off between the standard communication-heavy JCG and the Pipelined JCG variant, which reorganizes the algorithm to overlap global reductions with computation.

1.1 Datasets

The sparse SPD matrix A used for benchmarking are generated synthetically using a three-dimensional, 27-point finite difference stencil [10] on a structured cubic grid. As shown in Figure 1, each grid point represents a unique unknown and a corresponding row in the matrix A . In this 27-point configuration, each row contains up to 27 non-zero entries, as

every point couples to all 26 neighbors in its $3 \times 3 \times 3$ vicinity. We evaluate grid sizes up to 1100^3 (approximately 1.33 billion unknowns), stressing both memory bandwidth and inter-node synchronization.

1.2 Sequential Algorithm Analysis and Pseudo-code

The standard JCG (Algorithm 1) is computationally efficient but requires two sequential synchronization points per iteration for dot products, forcing processors to idle. In contrast, the Pipelined JCG (Algorithm 2) introduces auxiliary vectors to allow the global reduction to overlap with the matrix-vector product (SpMV) and halo exchanges.

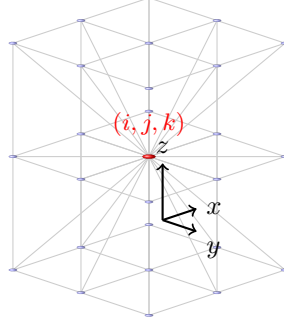


Figure 1: The 27-point finite difference stencil. The central node (red) is coupled to all 26 neighbors in the $3 \times 3 \times 3$ vicinity, including face-sharing, edge-sharing, and corner-sharing nodes.

To understand the necessity of parallelization and pipelining, we perform a detailed analysis of the computational and memory requirements of the sequential JCG algorithm. Both implemented variants employ a Jacobi preconditioner $\mathbf{M} = \text{diag}(\mathbf{A})$, where $\mathbf{M}$ is a symmetric positive-definite approximation of $\mathbf{A}$, which clusters eigenvalues to accelerate convergence with negligible computational overhead [11].

1.2.1 Storage and Computational Complexity

To handle the scale of the 3D grid, the matrix $\mathbf{A}$ is stored in the Compressed Sparse Row (CSR) format. This reduces memory requirements from $\mathcal{O}(n^2)$ to $\mathcal{O}(\text{nnz})$, where $\text{nnz} \approx 27n$, enabling the solution of systems with over 1.33×10^9 unknowns. The complexity of each iteration is broken down in table 1.

Table 1: Per-iteration floating-point cost comparison of Standard and Pipelined JCG.

Operation	Standard JCG	Pipelined JCG	Difference
SpMV (27-pt)	53	53	0
Preconditioner	1	1	0
Vector Arithmetic	10 (3 AXPY + 2 Dot)	20 (8 AXPY + 2 Dot)	+10
Total FLOPs/	64	74	+10

While the pipelined variant increases arithmetic work by 16%, it is a deliberate trade-off to allow the global reduction latency ($\mathcal{O}(\log P)$) to be overlapped with the SpMV and halo exchanges in parallel computation.

1.2.2 Arithmetic Intensity of the SpMV Kernel

The 27-point SpMV is strongly memory-bound. Applying the Roofline Model [14], each grid point performs 53 FLOPs while transferring 30 double-precision values (27 matrix coefficients, one input vector read, and two output vector read/writes). The operational intensity (OI) is:

$$OI_{\text{SpMV}} = \frac{53}{8 \times (27 + 3)} \approx 0.22 \text{ FLOPs/Byte.} \quad (1)$$

This 0.22 FLOPs/Byte is significantly below the machine balance of modern processors, such as the Intel Xeon 6252N (exceeds 7.0 FLOPs/Byte [1]), confirming that performance is limited by memory bandwidth rather than peak throughput. This observation justifies the Pipelined JCG approach: the computational tax of roughly 10 additional vector FLOPs per iteration utilizes otherwise idle CPU cycles during memory stalls. By inserting these operations into

memory-bound phases, Pipelined JCG effectively masks global communication latency (G) within the SpMV execution, improving scalability without increasing total wall-clock time.

Algorithm 1 Jacobi Preconditioned Conjugate Gradient (JCG)

Require: A SPD, b , M^{-1} , x_0 , tolerance tol , max_iter
0: $r_0 \leftarrow b - Ax_0$, $u_0 \leftarrow M^{-1}r_0$, $p_0 \leftarrow u_0$
0: **for** $i = 0$ to $max_iter - 1$ **do**
0: $s_i \leftarrow Ap_i$
0: $\alpha_i \leftarrow (r_i, u_i) / (s_i, p_i)$
0: $x_{i+1} \leftarrow x_i + \alpha_i p_i$
0: $r_{i+1} \leftarrow r_i - \alpha_i s_i$
0: $u_{i+1} \leftarrow M^{-1}r_{i+1}$
0: $\beta_i \leftarrow (r_{i+1}, u_{i+1}) / (r_i, u_i)$
0: $p_{i+1} \leftarrow u_{i+1} + \beta_i p_i$
0: **if** $\frac{\|r_k\|_2}{\|r_0\|_2} < tol$ **then**
0: **break**
0: **end if**
0: **end for**=0

Algorithm 2 Jacobi Preconditioned Pipelined CG (Pipeline JCG)

Require: A SPD, b , M^{-1} , x_0 , tolerance tol , max_iter
0: $r_0 \leftarrow b - Ax_0$, $u_0 \leftarrow M^{-1}r_0$, $w_0 \leftarrow Au_0$
0: **for** $i = 0$ to $max_iter - 1$ **do**
0: $c_i \leftarrow (r_i, u_i)$
0: $d_i \leftarrow (w_i, u_i)$
0: $m_i \leftarrow M^{-1}w_i$, $n_i \leftarrow Am_i$
0: **if** $i > 0$ **then**
0: $\beta_i \leftarrow c_i / c_{i-1}$
0: $\alpha_i \leftarrow c_i / (d_i - \beta_i c_i / \alpha_{i-1})$
0: **else**
0: $\beta_i \leftarrow 0$, $\alpha_i \leftarrow c_i / d_i$
0: **end if**
0: $z_i \leftarrow n_i + \beta_i z_{i-1}$
0: $q_i \leftarrow m_i + \beta_i q_{i-1}$
0: $s_i \leftarrow w_i + \beta_i s_{i-1}$
0: $p_i \leftarrow u_i + \beta_i p_{i-1}$
0: $x_{i+1} \leftarrow x_i + \alpha_i p_i$
0: $r_{i+1} \leftarrow r_i - \alpha_i s_i$
0: $u_{i+1} \leftarrow u_i - \alpha_i q_i$
0: $w_{i+1} \leftarrow w_i - \alpha_i z_i$
0: **if** $\frac{\|r_k\|_2}{\|r_0\|_2} < tol$ **then**
0: **break**
0: **end if**
0: **end for**=0

2 Parallel Design

The parallelization of the JCG method for large-scale 3D systems requires a transition from a mathematically minimal sequential formulation to a communication-hiding parallel architecture. This section analyzes the inherent data dependencies of the algorithm and selects the data structure that minimizes the synchronization burden on distributed hardware, evaluates alternative parallel strategies, and motivates the selection of the Pipelined JCG variant.

2.1 Analysis of Parallelism and Algorithmic Dependencies

The iterative nature of the Conjugate Gradient method imposes a strict temporal dependency, where each iteration k requires the search direction and residual from $k - 1$, which creates an inherent sequential dependency between iterations. Consequently, iterations are inherently sequential and cannot be executed concurrently[11]. Parallelism must therefore be exploited within each iteration, imposing a fundamental limit on achievable scalability as stated by Amdahl's Law that the speedup of a parallel program is limited by its strictly sequential fraction. Vector updates(AXPY) and the application of the Jacobi preconditioner exhibit ideal data parallelism; since these operations are element-wise, they can be executed with zero inter-process communication provided that the vectors are partitioned identically across the MPI processes.

In contrast, the Sparse Matrix-Vector product (SpMV) and global inner products represent the primary bottlenecks to scalability. In the context of a 27-point stencil, each grid point calculation requires access to 26 neighboring values of the search direction vector p . In a distributed memory environment, these neighbors may reside on different MPI processes, necessitating a strategy for non-local data access. In this work, we analyze two distinct strategies for satisfying this dependency:

1. **Global Replication (Baseline):** The entire vector p is assembled on every process using collective communication (MPI_Allgather), allowing the SpMV to proceed as a local operation at the cost of $O(n)$ communication volume.
2. **Halo Exchange (Pipelined):** Processes exchange only the boundary ‘halo’ planes required by the stencil via point-to-point communication (3), reducing the data volume but increasing implementation complexity.

Furthermore, global dot products require collective reductions (e.g., MPI_Allreduce), which act as hard synchronization points. These operations force all processes to stall until a global consensus is reached, meaning that network latency—rather than computational speed—becomes the dominant factor in total execution time as the process count increases [11]. The primary motivation for the Pipelined JCG variant is to mathematically re-arrange these dependencies, allowing these synchronization costs to be hidden behind the SpMV kernel.

2.2 Domain Decomposition: Z-Slab Partitioning

To distribute the computational load across P MPI processes, we adopt a Z-slab domain decomposition (Figure 2). The global grid of size $N_x \times N_y \times N_z$ is partitioned along the Z -axis, assigning each process a sub-domain of dimensions $N_x \times N_y \times N_{\text{loc}}$, where $N_{\text{loc}} \approx N/P$.

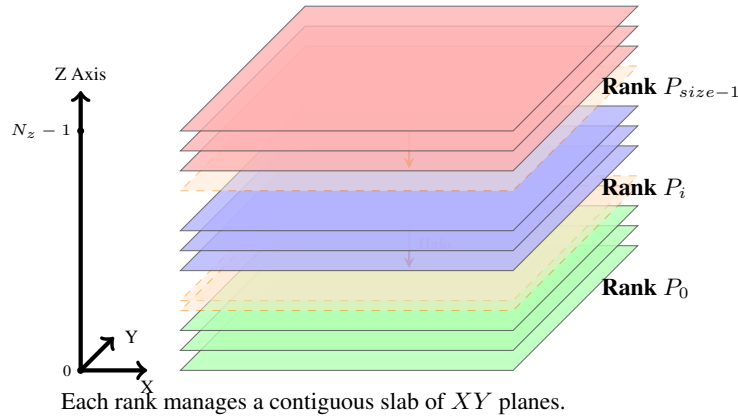


Figure 2: Z-Slab domain decomposition. The 3D grid is partitioned along the Z -axis, ensuring that XY planes are contiguous in memory for each MPI rank, facilitating efficient communication of boundary planes.

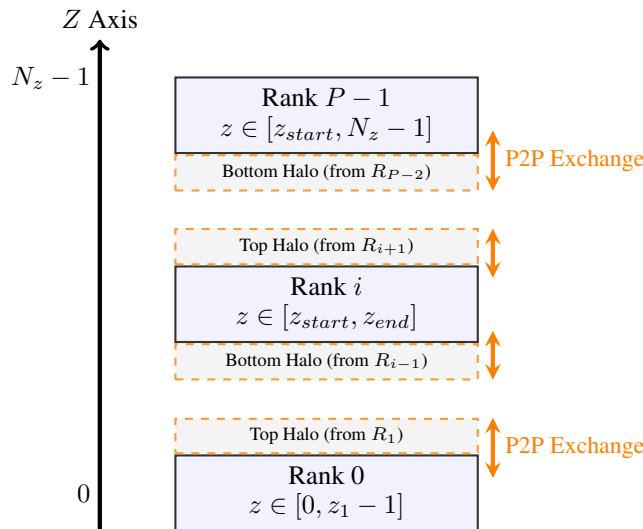


Figure 3: Communication topology for the Pipelined JCG. Ranks are organized in a vertical 1D chain corresponding to the Z-slab decomposition. Each process exchanges only the boundary XY -planes with its vertical neighbors, significantly reducing communication volume compared to global replication (i.e. p) in JCG.

This decomposition is preferred over a 3D block (sub-cube) partitioning for two reasons. First, it produces a simple 1D communication topology in which each interior rank exchanges data with only its immediate "up" and "down" neighboring plan. Second, because the data is stored in row-major order (X fastest, then Y , then Z), the boundary XY -planes between slabs are contiguous in memory. Because of that it can be transferred with a single MPI call, eliminating the overhead of packing non-contiguous data or constructing MPI derived datatypes.

2.3 Parallel Strategies and State of the Art

Scalability of the CG method is limited by synchronization overhead, as global reductions (dot products) act as hard barriers in distributed memory. State-of-the-art solvers address this through communication-hiding and communication-avoiding techniques. Pipelined CG overlaps reductions with independent computations (SpMV and preconditioning), effectively shifting the bottleneck from network latency to local memory bandwidth [3]. In contrast, s -step CG reduces the frequency of global synchronization by performing multiple iterations simultaneously, at the cost of increased memory traffic and potential numerical instability [2].

In this work, we compare two parallel JCG architectures. The baseline approach employs full vector replication using `MPI_Allgatherv`, simplifying SpMV to a local operation but incurring $O(n)$ communication and non-scalable memory usage. The Pipelined JCG instead uses nearest-neighbor halo exchanges with a Z-slab decomposition, by partitioning the grid into horizontal planes, communication is restricted to immediate "up" and "down" neighbors, reducing the memory footprint to $O(n/P)$ and allowing for a halo-aware split-SpMV kernel.

These strategies are implemented in HPC libraries: PETSc provides classic and pipelined PCG [8, 9], Trilinos offers CG solvers via Belos [12, 4], and Hypr includes parallel solvers and preconditioners for large-scale sparse systems [7, 6]. Preconditioning remains crucial, as local methods like Jacobi are fully parallelizable and can reduce total iteration counts [6].

2.4 Architectural Trade-offs: The Computational Tax

A key aspect of the parallel design is the intentional increase in computational work to enable communication overlap. While the standard formulation requires roughly 64 FLOPs per grid point, the Pipelined JCG variant raises this to about 74 FLOPs. This additional cost, the 'computational tax', results primarily from the introduction of auxiliary vectors (w, u, q, z) , which increase the number of vector updates from 3 to 8.

This trade-off is justified by the arithmetic intensity of the 27-point stencil ($AI \approx 0.22$). Because the SpMV kernel is strongly memory-bound, the CPU frequently stalls waiting for data from main memory. The extra 10 FLOPs are inserted into these otherwise idle cycles, allowing the algorithm to break the critical-path dependencies associated with global dot products without increasing per-iteration wall-clock time. As a result, the solver shifts from being latency-bound to bandwidth-bound, a transition that is crucial for scaling to large problem sizes such as the 1100^3 grid.

2.5 Proposed Hybrid Parallelization and Data Dependencies

To maximize utilization of many-core nodes, our design adopts a hybrid MPI+OpenMP strategy. This approach manages data dependencies across the memory hierarchy to enable communication-computation overlap.

2.5.1 Shared-Memory Level (OpenMP)

Inside each MPI rank, we parallelize the 27-point stencil across the local Z-slab. Following the scoping principles, we identify the following dependencies:

- **SpMV Hazards:** In the operation $s = Ap$, the input vector p is shared and read-only, while the output vector s is shared but uniquely partitioned by index. This elimination of Read-After-Write (RAW) hazards allows for synchronization-free thread execution.
- **Reduction Pattern:** For dot products, we propose a two-stage reduction. Threads first perform a local reduction to a private variable, which is then aggregated into the rank-level scalar using a critical section or the reduction clause. This minimizes the synchronization floor before entering the MPI layer.

2.5.2 Distributed-Memory Level (MPI)

The core of our design is breaking the *temporal dependency* of the standard JCG. In the pipelined variant, the scalar α_i is required only at the end of the iteration, allowing for an Asynchronous Split-Kernel design:

1. **Initiation:** Trigger non-blocking MPI_Iallreduce (for scalars) and MPI_Isend/MPI_Irecv (for halos).
2. **Internal SpMV:** Threads compute grid points that do not require non-local data. This phase hides the global reduction latency.
3. **Boundary SpMV:** After an MPI_Wait on halos, threads compute the slab boundaries.
4. **Final Sync:** Retrieve the reduction result to update the search direction for the next iteration.

This 4-step sequence is executed concurrently by each MPI process. Within each process, Steps 2 and 3 are parallelized using `#pragma omp parallel for`, effectively delegating the stencil computation to worker threads while the MPI subsystem handles non-blocking communication in the background.

This theoretical model shifts the implementation from a latency-bound regime to a bandwidth-bound regime, maximizing the 0.22 FLOPs/Byte operational intensity of the 3D stencil.

3 Implementation

The solver is implemented in C using the MPI-3.2 standard. The project follows a modular architecture separating grid generation, sparse storage, and solver logic. The complete source code and benchmarking scripts are available in the project repository.¹

3.1 Matrix Representation and Domain Decomposition

To achieve memory scalability, the 3D grid is partitioned using a Z-slab decomposition (Figure 2). Each MPI rank independently generates its local block of the 27-point stencil matrix in CSR format. The local unknown count is balanced as

$$n_{\text{loc}} = n_x n_y \left(\left\lfloor \frac{n_z}{P} \right\rfloor + \text{remainder} \right).$$

This localized construction avoids the $O(N^3)$ memory bottleneck of a centralized generator, ensuring the footprint per rank remains $O(N^3/P)$. To facilitate verification, a synthetic RHS is constructed as

$$b = Ax_{\text{known}}, \quad x_{\text{known}} = 2.0.$$

The CSR structure also precomputes the inverse diagonal values used by the Jacobi preconditioner.

Listing 1: CSR Structure and Split-SpMV Kernel

```
typedef struct
{
    int n;           // Global number of rows
    int n_local;     // Number of local rows
    int row_start;   // Global row start index
    int nnz;         // Number of non-zero entries (local)
    int *row_ptr;    // CSR row pointers
    int *col_indices; // CSR column indices
    double *values;  // CSR non-zero values
    double *inv_diag; // Inverse of diagonal elements (for Jacobi preconditioning)
} CSR;

// Prepare Halo Exchange for m0 while computing internal SpMV
halo_exchange_init(G, m, r_up, r_down, h_req);
// SpMV: v0 = A * m0 (internal + boundary)
csr_sparse_matvec_mult_internal(A, G, m, v);
MPI_Waitall(4, h_req, MPI_STATUSES_IGNORE);
csr_sparse_matvec_mult_boundary(A, G, m, r_up, r_down, v);
```

¹<https://github.com/datrento/parallel-conjugate-gradient>

3.2 Solver Architectures and Overlap Logic

The project compiles two distinct binaries from the same source tree using compile-time flags to evaluate different communication strategies:

- **Baseline JCG (Global Replication):** This variant utilizes `MPI_Allgatherv` to assemble the full search direction p on every rank before a standard local SpMV. While the implementation is simple, the $O(N)$ communication volume and memory overhead per rank prevent execution on the largest grids such as 1100^3 .
- **Pipelined JCG (Halo Exchange & Overlap):** This variant utilizes nearest-neighbor halo exchanges ($O(N^2)$ volume) and non-blocking `MPI_Iallreduce` to hide synchronization latency. By employing the split-kernel logic (Listing 1), internal grid points are computed while halo data and global scalars are in flight.

The build system and benchmarking scripts automate the execution of these binaries across various grid sizes and process counts. Finally, correctness is validated after each run by verifying that the computed solution satisfies $Ax \approx b$.

3.3 Experimental Setup

To ensure reproducibility and characterize the peak performance potential of the solvers, every experiment was conducted with three independent repetitions, consistent with standard high-performance computing benchmarking practices, we report the minimum execution time across the three trials for all scaling analysis. By selecting the minimum, we effectively filter out external system-induced delays and isolate the algorithmic efficiency of the solvers under optimal system conditions. This approach allows for a clearer evaluation of the strong and weak scaling characteristics without the distortion of non-deterministic cluster noise.

All experiments were conducted on the University of Trento HPC cluster using the `short_HPC4DS` partition. The hardware consisted of Intel(R) Xeon(R) Gold 6252N CPUs (96 cores per node) with a high-speed interconnect. We evaluated five grid resolutions to capture performance across different scales as outlined in Table 2. Due to the architectural memory limitations of the baseline approach (the $O(N)$ memory wall), the 645^3 , 813^3 , 875^3 , and 1100^3 baseline results were obtained from 21-iteration steady-state throughput probes to ensure a fair comparison within system walltime limits.

Table 2: Key experimental parameters for the PCG solvers.

Parameter	Value / Range
Grid dimensions (N^3)	$400^3, 512^3, 645^3, 813^3, 875^3, 1100^3$
MPI Ranks (P)	96, 192, 384 (1, 2, 4 nodes)
Stencil Type	27-point 3D Finite Difference
Residual Tolerance	1×10^{-10}

3.4 Performance Metrics

To mitigate the impact of transient system noise and interconnect jitter, we quantify parallel performance using the wall-clock time (T_P), defined as the minimum of the maximum solver durations recorded across all ranks over three independent trials.

Absolute Speedup (S_P^{abs}) Absolute gain compared to the 1-node Baseline JCG: $S_P^{\text{abs}} = T_{96}^{\text{Base}} / T_P^{\text{Algo}}$.

Self-Scaling Speedup (S_P^{self}) Internal scalability relative to the algorithm’s own 1-node time: $S_P^{\text{self}} = T_{96}^{\text{Algo}} / T_P^{\text{Algo}}$.

Parallel Efficiency (E_P) Resource utilization normalized by node count ($N = P/96$): $E_P = (S_P^{\text{self}} / N) \times 100\%$.

Weak Scaling Efficiency (W_P) Throughput stability at a constant workload ($\approx 1.4 \times 10^6$ unknowns/rank) using average iteration time (T_{iter}): $W_P = (T_{\text{iter}, 96} / T_{\text{iter}, P}) \times 100\%$.

3.5 Numerical Validation and Convergence

Before analyzing the computational performance, we verified that the pipelined transformation maintained the numerical integrity of the Conjugate Gradient method. Figure 4 illustrates the residual norm reduction

$$\frac{\|r_k\|_2}{\|r_0\|_2}$$

for both the baseline and pipelined solvers on the 512^3 grid.

The convergence curves for both algorithms overlap perfectly, with both solvers reaching the 1×10^{-10} relative tolerance in approximately 206 iterations. This numerical equivalence is a critical result; it demonstrates that the pipelined reformulation—while significantly altering the execution order to allow for communication hiding—does not introduce numerical instability or alter the convergence rate. Consequently, the performance gains reported in the following sections are purely a result of improved communication–computation overlap and reduced global synchronization overhead.

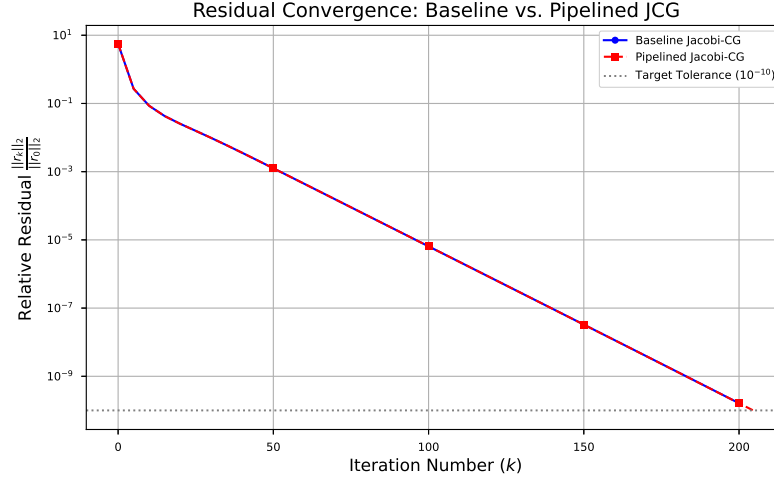


Figure 4: Residual convergence for Baseline and Pipelined PCG. The perfect overlap confirms that the pipelined reformulation is numerically equivalent to the standard algorithm.

4 Result Analysis

4.1 Strong Scalability

Strong scalability assess how well the implementation reduces execution time as the number of processing elements increases for a fixed total problem size. In the context of Jacobi-PCG, scalability is primarily limited by the overhead of collective communications and the saturation of the memory subsystem.

4.1.1 Intra-Node Scale-Up Analysis

We first evaluated the strong-scaling performance within a single Xeon 6252N node using a grid size of $N = 400$ (64×10^6 unknowns). This configuration represents the largest 27-point stencil problem that allows a reliable serial baseline while remaining within 32-bit addressing limits on a single rank. As shown in Figure 5, the two algorithms diverge sharply in scalability.

The baseline PCG suffers a severe performance collapse beyond 16 cores: execution time rises from 96.8 s to 261.2 s at 96 cores. This negative scaling is driven by the $\mathcal{O}(n)$ global assembly of the search direction vector, implemented through synchronous collective operations. Within a single node, these collectives induce memory contention and synchronization stalls that rapidly overshadow the benefits of additional parallelism. In contrast, the pipelined PCG completes the solve in 27.3 s at 96 cores, achieving a $9.5\times$ speedup over the baseline.

The performance advantage of the pipelined solver arises from its restructuring of communication in the 3D domain decomposition. In the baseline algorithm, each iteration performs a global `MPI_Allreduce` for the dot product followed by an `MPI_Allgatherv` to assemble the full search direction vector of size N^3 , because each CSR matrix references global indices. These sequential collectives introduce serialization points where fast ranks idle until the slowest rank completes. The pipelined variant removes the `MPI_Allgatherv` entirely by expressing the SpMV in terms of local data plus halo regions. After issuing a non-blocking `MPI_Iallreduce`, each rank exchanges only a single xy -plane of halo data with its immediate z -neighbors, reducing communication from $\mathcal{O}(N^3)$ to $\mathcal{O}(N^2)$. While these halo exchanges progress asynchronously, the rank computes the SpMV on interior points and later completes the boundary updates

once halos arrive. By the time the dot-product result is needed via `MPI_Wait`, both the $\mathcal{O}(\log P)$ reduction and the $\mathcal{O}(N^2)$ halo transfer typically complete, effectively hiding communication under the $\mathcal{O}(N^3/P)$ local computation.

Despite the pipelined solver’s advantages, both algorithms encounter a memory-bandwidth wall between 32 and 48 cores, where the 27-point stencil becomes limited by data movement rather than computation. Once the six memory channels of the Xeon processor are saturated, further cores offer diminishing returns. Under these conditions, the pipelined variant maintains stable performance, whereas the baseline suffers additional overhead from its global vector assembly, underscoring its unsuitability for high-density stencil computations even at the intra-node scale.

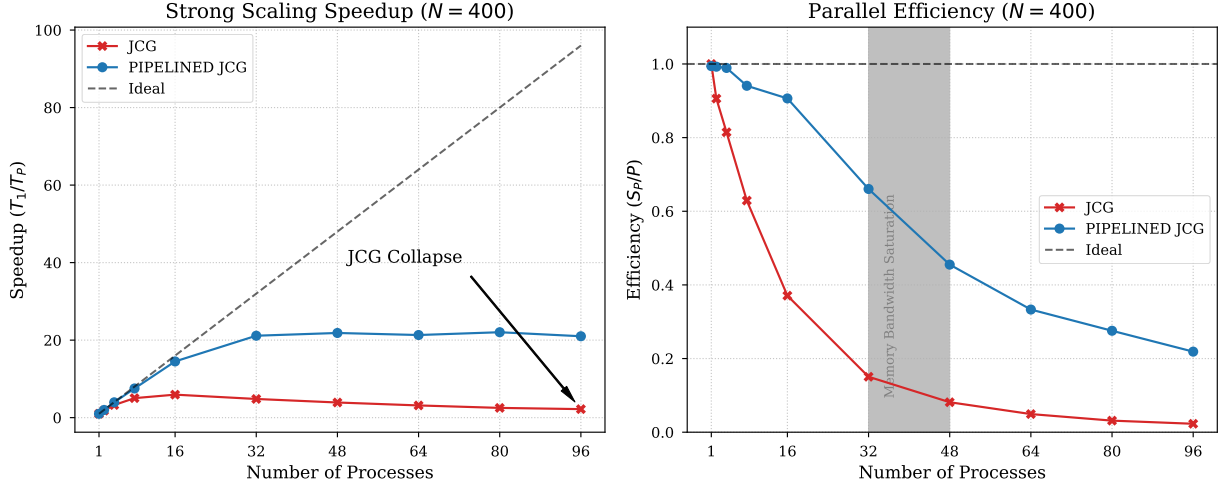


Figure 5: Strong scaling analysis for $N = 400$ grid size. The pipelined variant (blue) significantly outperforms the baseline (red) at high core counts by mitigating global synchronization costs.

4.1.2 Inter-Node Strong Scaling

To overcome the memory bandwidth saturation observed within a single node, we scale the problem out to a multi-node configuration (up to 4 nodes). To evaluate the impact of network latency and the effectiveness of communication hiding at a larger scale, we performed inter-node strong scaling across 1, 2, and 4 nodes (up to 384 MPI ranks). For these experiments, we utilized three grid sizes (N): 512^3 , 875^3 , and 1100^3 , the latter representing the "Highest Size" problem with over 1.33 billion unknowns. The execution times and projected performance are summarized in Table 3.

The intra-node strong scaling analysis established that both JCG and pipelined JCG encounter a memory-bandwidth wall between 32 and 48 cores, where performance becomes limited by data movement rather than computation due to the 27-point stencil. Building on this, the inter-node strong scaling analysis isolates the impact of the network interconnect from intra-node memory effects by reporting node-relative speedup $S_P = T_{96}/T_P$, where execution times on two and four nodes are normalized to the single-node (96-rank) JCG reference. As the number of nodes increases, the pipelined JCG algorithm maintains significantly higher speedup and throughput compared to JCG, particularly for larger problem sizes. This demonstrates that, beyond the intra-node memory wall, the pipelined approach is more resilient to inter-node communication bottlenecks, while JCG suffers from poor scalability due to increased global synchronization overheads.

For the largest resolutions (875^3 and 1100^3), where the baseline solver frequently exceeded system walltime and memory limits, performance was estimated using a linear performance extrapolation model. This approach is grounded in the observation that Conjugate Gradient solvers are strictly memory-bandwidth limited, meaning the time per iteration is dictated by the ratio of invariant data volume to sustained memory throughput [13]. Consequently, once the system reaches a steady state [10, 3], the execution time per iteration remains essentially constant. Following the High Performance Conjugate Gradients (HPCG) benchmark methodology for subset timing [5], average steady-state iteration times ($\bar{T}_{iter}$) were captured over 21-iteration probes to average out transient system noise. By accounting for the solver’s initialization overhead (T_{setup}) and extrapolating $\bar{T}_{iter}$ by the converged iteration counts (N_{conv}) of the numerically equivalent pipelined variant [3], which we observed to take *orinus1* number of iteration differences, the total time-to-solution was projected as follows:

$$T_{total} = T_{setup} + N_{conv} \times \bar{T}_{iter} \quad (2)$$

For strong scaling analysis, we compare the baseline JCG and the pipelined JCG algorithms in terms of speedup and parallel efficiency. Speedup is computed relative to the baseline JCG at 1 node (96 ranks) wherever possible, providing a direct measure of the performance improvement achieved by the pipelined approach. For problem sizes where the baseline JCG could not be run at 1 node due to memory limitations (e.g., $N = 1100^3$), speedup for the pipelined JCG is referenced to its own 1-node result. Parallel efficiency is always computed relative to each algorithm’s own 1-node performance. This isolates the scalability of each method independently of its absolute runtime differences, allowing a fair assessment of how well each algorithm utilizes additional resources as the number of nodes increases.

As shown in Figure 6, the left panel presents speedup computed relative to the baseline JCG at one node (96 ranks), enabling a direct performance comparison between the two algorithms. Under this normalization, the separation between the baseline and pipelined speedup curves quantifies the performance advantage of the pipelined formulation.

For the 512^3 problem, the baseline JCG exhibits limited scalability, achieving speedups of $1.03\times$ and $0.92\times$ on 2 and 4 nodes, respectively. In contrast, the pipelined JCG reaches $13.41\times$, $23.18\times$, and $33.65\times$ on the same node counts. At four nodes, this corresponds to a baseline-normalized speedup increase of roughly 33-fold, clearly illustrating the benefit of communication–computation overlap.

A similar trend is observed for the 875^3 grid. The baseline JCG attains only $1.09\times$ and $1.14\times$ on 2 and 4 nodes, whereas the pipelined JCG achieves $13.07\times$, $23.35\times$, and $37.49\times$ on 1, 2, and 4 nodes. At four nodes, the baseline-normalized speedup gap is approximately $37.49 - 1.14 \approx 36.3$, showing that the pipelined formulation provides over an order-of-magnitude improvement compared to the standard JCG under the same reference model.

For the largest problem size (1100^3), the baseline JCG fails to run on one and two nodes due to memory exhaustion, so only the pipelined JCG is reported, referenced to its own 1-node performance. This highlights the memory-bound nature of the baseline solver and the necessity of communication-hiding and reduced-memory techniques at scale.

For evaluation of parallel efficiency and scaling trends independently, each algorithm is normalized to its own 1-node performance. This approach allows assessment of how well each method utilizes additional nodes regardless of differences in absolute runtime. Using this metric, the pipelined JCG sustains high efficiency across all problem sizes and node counts, whereas the baseline JCG shows rapid efficiency degradation, particularly for the larger grids. For instance, the JCG algorithm shows relative speedups ranging only from $0.92\times$ to $1.13\times$ on four nodes, corresponding to parallel efficiencies as low as 23%. This breakdown is caused by the synchronous dependency of the $O(N^3)$ global vector assembly and global reductions (e.g., dot products), which require all processes to synchronize through collective operations such as `MPI_Allgather` and `MPI_Allreduce` before the stencil application can proceed. The resulting processor idling limits throughput to only 0.03 iterations per second at the 1100^3 scale (Figure 7 and 3).

Table 3: Comparison of total time to solution (seconds). For $N \geq 875$, JCG values are projected based on the converged iteration counts of the numerically equivalent pipelined solver to facilitate a consistent comparison.

Grid (N^3)	Algorithm	1 Node (96)	2 Nodes (192)	4 Nodes (384)
512	JCG	666.44	649.91	724.08
	Pipelined JCG	49.45	28.62	19.71
875	JCG*	3016.62	2756.50	2653.07
	Pipelined JCG	230.79	129.18	80.47
1100	JCG*	OOM	OOM	6746.15
	Pipelined JCG	451.58	240.35	128.54

*Projected based on 21-iteration steady-state throughput probes.

OOM: Out-of-memory termination (Signal 9) occurred during allocation.

In contrast, the pipelined JCG demonstrates strong internode scalability. For the largest feasible grid ($N = 1100^3$), the baseline JCG could not be run at 1 or 2 nodes due to out-of-memory errors, so speedup and efficiency are referenced to the pipelined JCG at 1 node. On four nodes, the pipelined JCG achieves a speedup of $3.51\times$ and a parallel efficiency of 87.8%. For smaller grids, such as $N = 875^3$ and $N = 512^3$, the pipelined JCG achieves speedups of $2.87\times$ (72% efficiency) and $2.51\times$ (63% efficiency) on four nodes, respectively, each relative to its own 1-node performance.

In contrast, the pipelined JCG demonstrates strong internode scalability. For the largest feasible grid ($N = 1100^3$), it achieves a speedup of $3.51\times$ and parallel efficiency of 87.8% on four nodes. For smaller grids ($N = 875^3$ and

$N = 512^3$), the pipelined JCG reaches speedups of $2.87\times$ (72% efficiency) and $2.51\times$ (63% efficiency) on four nodes, respectively.

By reformulating the CG algorithm to support asynchronous communication–computation overlap [3], the solver effectively masks the latency of global reductions and halo exchanges behind the 27-point stencil computation. A key observation from the speedup analysis is the positive surface-to-volume trend: parallel efficiency increases from 63% at 512^3 to 87.8% at 1100^3 , indicating that larger local subdomains provide a wider computational window in which to hide internode communication.

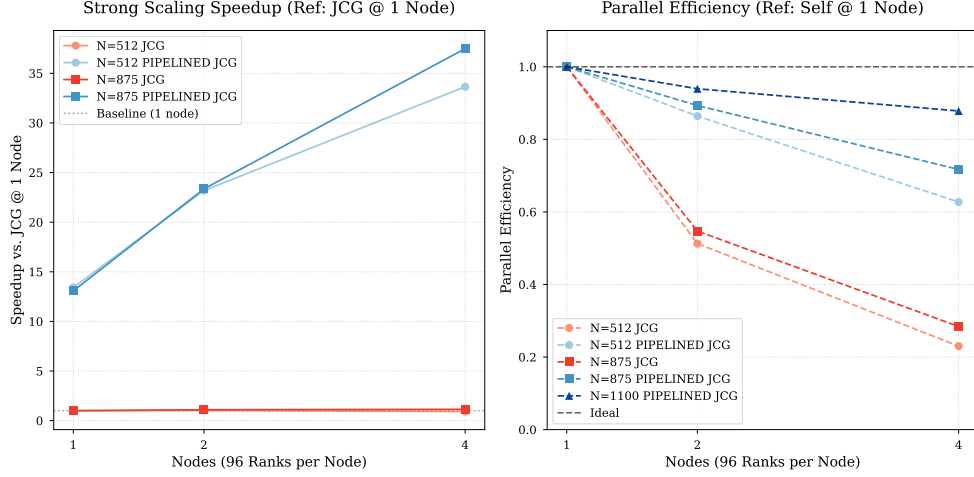


Figure 6: **Left:** Strong scaling speedup of JCG and pipelined JCG for problem sizes $N = 512$ and $N = 875$, referenced to JCG at 1 node (96 ranks). The horizontal dashed line marks the JCG performance at 1 node. No ideal linear speedup line is included because the baseline is already parallel. **Right:** Parallel efficiency of both algorithms, each normalized to its own performance at 1 node. The pipelined JCG sustains higher efficiency across increasing node counts, particularly for the larger problem size, indicating improved strong-scaling behavior compared to JCG.

Overall, these findings show that at the billion-unknown scale, the pipelined formulation is not merely a performance optimization but a structural requirement for running CG solvers on memory-constrained HPC systems. Unlike the baseline JCG, which encounters a memory wall at the 1100^3 resolution—triggering out-of-memory terminations due to its $O(N)$ storage per rank—the pipelined solver maintains an $O(N/P)$ memory footprint, enabling stable execution while also delivering substantially higher scalability and throughput.

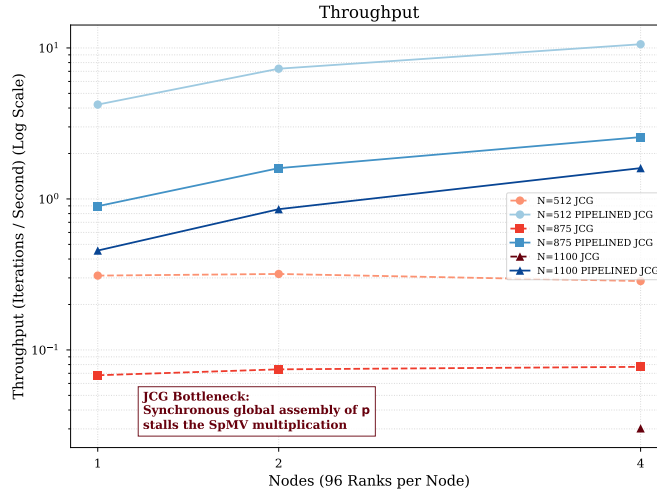


Figure 7: Throughput (iterations per second, log scale) of JCG and pipelined JCG for problem sizes $N = 512$, $N = 875$, and $N = 1100$, shown as a function of node count (96 ranks per node).

4.1.3 Weak Scaling Analysis

To evaluate the weak scalability of both solvers, we analyzed performance as the problem size and the number of processes increased proportionally, maintaining a roughly constant workload per process—specifically, about 1.4 million degrees of freedom (DOF) per MPI rank. For each configuration, the global grid size was selected to ensure that the total number of unknowns scaled linearly with the number of processes, allowing a fair assessment of parallel overheads as the system grows.

To assess weak scaling, we focus on the average iteration time as the primary metric. This is standard practice because, under ideal weak scaling, the time required for each iteration should remain constant as both the problem size and the number of processes increase, provided the workload per process does not change. Any increase in average iteration time directly reflects parallel overheads, such as communication and synchronization costs, rather than increased computational work per process. Therefore, parallel efficiency is computed as the ratio of the average iteration time on one node to that on multiple nodes:

$$\text{Efficiency} = \frac{\text{Iteration time at 1 node}}{\text{Iteration time at } P \text{ nodes}}$$

Deviations from ideal efficiency are primarily caused by communication overhead, including both the blocking and non-blocking collective operations for gathering the search direction vector p (such as `MPI_Allgatherv`) and the global reductions required for inner products.

To evaluate the weak scalability of both solvers, we analyzed performance as the problem size increased from 134 million unknowns on 96 ranks (512^3) to 670 million unknowns on 384 ranks (813^3). Using the same methodology we used in the strong scaling, for the baseline algorithm at the 875^3 scale, total time was extrapolated from 21-iteration throughput measurements, since the $O(N^3)$ communication overhead of the `MPI_Allgatherv` operation made a full execution to convergence computationally infeasible within the system walltime limits.

As shown in Table 4 and Figure 8, the pipelined JCG maintains nearly constant average iteration time as both grid size and node count increase, demonstrating excellent weak scalability. In contrast, the baseline JCG exhibits a marked increase in average iteration time and a corresponding drop in efficiency, reflecting the growing impact of communication overheads at larger scales. On four nodes, the pipelined JCG achieves an efficiency of 79.7%, nearly double that of the baseline JCG (42.2%), and its average iteration time increases by only 25%, compared to a $2.4\times$ increase for the baseline solver.

Table 4: Average iteration time and parallel efficiency for baseline JCG and pipelined JCG (PJCG) under weak scaling, with approximately 1.4 million degrees of freedom (DOF) per process.

Nodes	Grid Size	Iter. Time (s)		Efficiency	
		JCG	PJCG	JCG	PJCG
1	512^3	3.6508	0.2381	1.000	1.000
2	645^3	4.8333	0.2678	0.755	0.889
4	813^3	8.6416	0.2985	0.422	0.797

These results confirm that, when grid size is chosen proportional to available resources, the pipelined JCG effectively hides communication costs and sustains high parallel efficiency, while the baseline JCG becomes increasingly dominated by global communication as the system grows.

4.2 Discussion

The experimental results reveal a stark divergence in the performance profiles of the baseline JCG and the pipelined JCG. This section analyzes the root causes of these differences, evaluates the effectiveness of communication-computation overlap, and discusses the architectural trade-offs observed during the study.

4.3 Performance Bottlenecks and Communication Overlap

To understand why the Pipelined JCG maintains high efficiency while the Baseline solver collapses, we analyze the iteration time breakdown illustrated in Figure 9. In the baseline case, visible communication overhead accounts for over 90% of the execution time at all scales. This represents the aggregated synchronization cost of the global data assembly (`MPI_Allgatherv`) and the global reductions (`MPI_Allreduce`) required for scalar updates. While our metrics report the total visible communication, the theoretical complexity suggests a dual bottleneck: the $O(n)$ data volume of

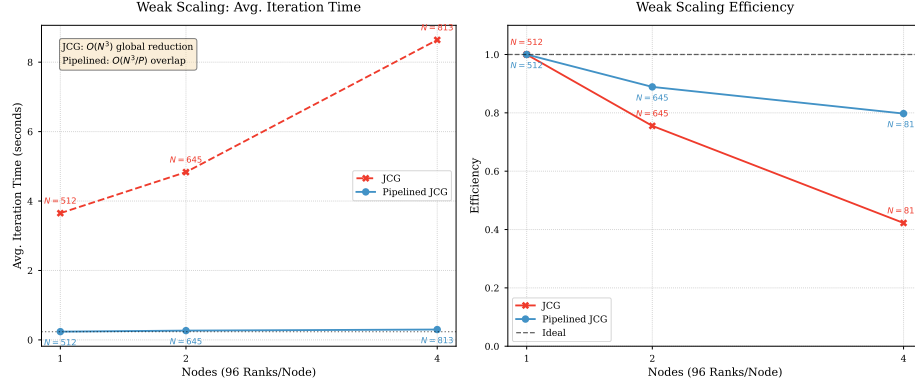


Figure 8: **Left:** Weak scaling of the average iteration time for the baseline JCG and pipelined JCG algorithms as problem size and node count increase proportionally. The pipelined JCG maintains nearly constant iteration time, while the baseline JCG shows growth due to communication overhead. **Right:** Corresponding weak scaling efficiency, normalized to 1 node. The pipelined JCG sustains high efficiency across nodes, whereas the baseline JCG efficiency drops significantly.

the vector replication saturates the interconnect bandwidth, while the blocking reductions impose a latency-driven synchronization barrier. Together, these collectives force the CPU into a stalled state for the vast majority of the iteration.

In contrast, the pipelined JCG successfully implements a communication-hiding strategy. While the solid bars (Computation) are slightly larger in the pipelined version—reflecting the 10-FLOP "computational tax" described in Section 2—the visible communication (hatched area) is nearly eliminated. This confirms our theoretical assessment based on the Roofline Model: with an arithmetic intensity of 0.22 FLOPs/Byte, the SpMV kernel provides a significant computational surplus. The pipelined algorithm utilizes these otherwise idle CPU cycles to calculate auxiliary vectors (w, u, q, z) while the global reductions and halo exchanges are processed by the network in the background.

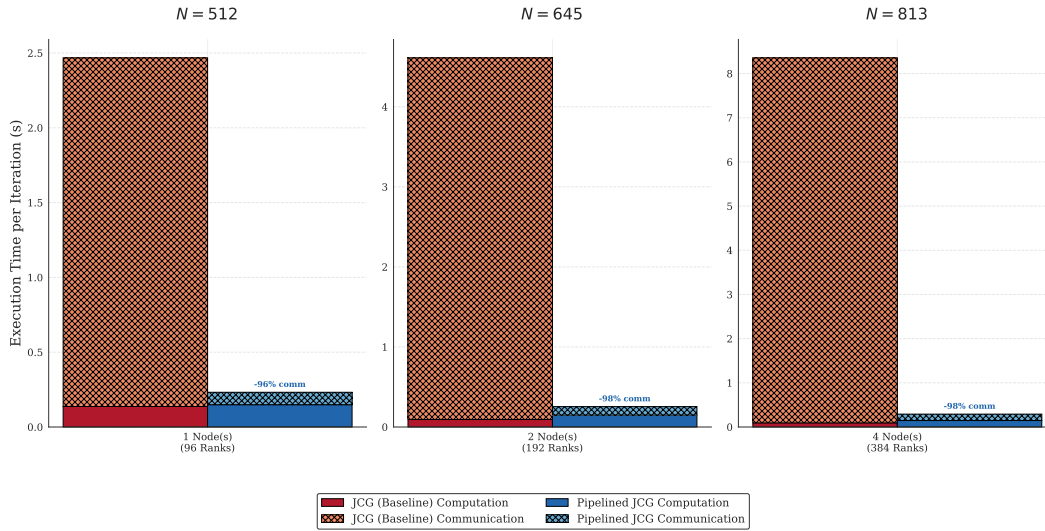


Figure 9: Iteration time breakdown between computation and visible communication for Baseline and Pipelined JCG. The hatched segments represent the time the CPU spent idling in MPI_wait. The Pipelined variant achieves near-total overlap, whereas the Baseline is entirely dominated by synchronization latency.

4.4 Memory Scalability and the "Memory Wall"

At the billion-unknown scale, the Pipelined formulation becomes a structural necessity rather than an optimization. The Baseline solver hits a "Memory Wall" at the 1100^3 resolution, terminating with Out-of-Memory (OOM) errors. This

limitation is inherent to its global replication strategy, which requires $O(n)$ storage per rank. At 1.33 billion degrees of freedom, a single vector occupies roughly 10.6 GB; duplicating this across 96 ranks exceeds node memory capacity.

In contrast, the Pipelined solver maintains an $O(n/P)$ memory footprint. By relying on nearest-neighbor halo exchanges instead of global assembly, it enables problem sizes that are infeasible under traditional approaches. This result demonstrates that communication-hiding and memory-distributed algorithms are essential for reaching “Grand Challenge” scale problems on modern HPC systems.

4.5 Limitations and Future Improvements

Despite its success, the current implementation (pipelined CG) has specific limitations:

- **Decomposition Rigidity:** The Z-slab decomposition requires that the number of Z -planes be at least equal to the number of MPI ranks. This restricts the maximum process count for “flat” grids and leads to high surface-to-volume ratios at extreme scales.
- **Load Balancing:** While the “remainder” distribution handles cases where n_z is not divisible by P , it only ensures balance to within one XY -plane. In more complex or irregular stencils, a more granular partition would be required.

4.6 Conclusion

This work demonstrates that the Pipelined Jacobi-CG variant significantly outperforms the standard formulation on modern HPC architectures. By paying a 16% computational tax in terms of additional vector arithmetic, we successfully transformed the solver from a latency-bound regime into a bandwidth-efficient implementation, consistent with the memory-bound limits predicted by our Roofline analysis. For our largest resolution (1100^3), the Pipelined JCG maintained 87.8% parallel efficiency across 4 nodes relative to its single-node performance, maintaining stable throughput where the Baseline solver failed. These results confirm that for large-scale 3D stencil problems, the combination of local CSR construction, halo-based communication, and overlapped non-blocking collectives provides a scalable and efficient path to the billion-unknown scale.

References

- [1] App metrics for intel® microprocessors. Technical report, Intel Corporation. Revision 7, Date: March 25, 2025.
- [2] A. T. Chronopoulos and C. W. Gear. s-step iterative methods for symmetric linear systems. *Journal of Computational and Applied Mathematics*, 1989.
- [3] P. Ghysels and W. Vanroose. Hiding global synchronization latency in the preconditioned conjugate gradient algorithm. 40(7):224–238.
- [4] Michael A. Heroux, Roscoe A. Bartlett, Vicki E. Howle, et al. An overview of the trilinos project. *ACM Transactions on Mathematical Software*, 2004.
- [5] HPCG Benchmark Project. Hpcg: High performance conjugate gradients benchmark, 2013.
- [6] hypr Developers. Boomeramg — hypr documentation.
- [7] hypr Developers. hypr documentation (pdf).
- [8] PETSc Developers. Kspcg: Preconditioned conjugate gradient (petsc manual page).
- [9] PETSc Developers. Kspipecg: Pipelined conjugate gradient method (petsc manual page).
- [10] Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- [11] Jonathan R Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. Technical report, USA, 1994.
- [12] Trilinos Project. Belos: Iterative linear solvers (documentation).
- [13] Mickeal Verschoor and Andrei C. Jalba. Analysis and performance estimation of the conjugate gradient method on multiple GPUs. 38(10):552–575.
- [14] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, April 2009.